

DESENVOLVIMENTO FULLSTACK

TRANSAÇÕES

(Diferentes métodos de classes distintas processados em uma mesma transação)

Podemos implementar o controle de transação centralizando a lógica da transação no nível do serviço. Utilizando uma classe para orquestrar os métodos separados, podemos implementar o controle das transações de modo que os métodos sejam processados em uma mesma transação.

Vamos abordar o controle de transação em um cenário onde existem métodos distintos para a inserção da Solicitação de compras (`solicitacao_compra`) e de seus itens (`itens_solicitacao`).

Estrutura geral

Suponha que no projeto temos:

1. **Método para inserir uma solicitação de compra**
(`inserirSolicitacaoCompra`)
2. **Método para inserir itens relacionados a uma solicitação**
(`inserirItensSolicitacao`)
3. **Controle de transação no nível do serviço**

O objetivo é garantir que tanto a inserção da solicitação quanto a inserção dos itens sejam realizadas como uma única transação.

Implementação

1. Gerenciando a conexão e a transação

É necessário criar um mecanismo para iniciar, confirmar ou reverter as transações.

Gerenciador de Transações

(`TransacaoBanco`):

```
const db = require('./db'); // Conexão ao banco de dados

class TransacaoBanco {
  constructor() {
    this.connection = null;
  }

  async iniciarTransacao() {
    this.connection = await db.getConnection(); // Obter uma conexão
    // específica
    await this.connection.beginTransaction(); // Iniciar a transação
  }

  async commit() {
    if (this.connection) {
```

```

        await this.connection.commit();
        this.connection.release(); // Liberar conexão
    }
}

async rollback() {
    if (this.connection) {
        await this.connection.rollback();
        this.connection.release(); // Liberar conexão
    }
}
}

module.exports = TransacaoBanco;

```

2. Métodos orientados a objetos separados

Criaremos as classes que representam os métodos para inserir Solicitações de Compras: `solicitacao_compra` e `itens_solicitacao`.

Classe para inserir as Solicitação de Compra

```

class SolicitarCompraService {
    constructor(conexao) {
        this.conexao = conexao;
    }

    async inserirSolicitacaoCompra(idUsuario) {
        const query = 'INSERT INTO SOLICITACAO_COMPRAS (ID_USUARIO) VALUES (?)';
        const [result] = await this.conexao.query(query, [idUsuario]); // Usando
a conexão específica
        return result.insertId; // Retorna o ID gerado
    }
}

```

Classe para inserir os Itens da Solicitação

```

class ItensSolicitacaoService {
    constructor(conexao) {
        this.conexao = conexao;
    }

    async inserirItensSolicitacao(idSolicitacao, itens) {
        const query = 'INSERT INTO ITENS_SOLICITACAO (ID_SOLICITACAO,
ID_PRODUTO, QUANTIDADE, PRECO_UNITARIO) VALUES (?, ?, ?, ?)';

        for (const item of itens) {
            await this.conexao.query(query, [
                idSolicitacao,
                item.id_produto,
                item.quantidade,
                item.preco_unitario
            ]);
        }
    }
}

```

3, Serviço principal para orquestrar a transação

No nível do serviço principal, implementaremos a transação e chamaremos os métodos separados.

Serviço principal

(OrquestradorSolicitacao)

```
const TransacaoBanco = require('./TransacaoBanco');
const SolicitarCompraService = require('./SolicitarCompraService');
const ItensSolicitacaoService = require('./ItensSolicitacaoService');

class OrquestradorSolicitacao {
  static async registrarSolicitacao(idUsuario, itens) {
    const transacao = new TransacaoBanco();

    try {
      // Iniciar transação
      await transacao.iniciarTransacao();
      const conexao = transacao.connection; // Obter conexão da transação

      // Instanciar serviços com a mesma conexão
      const solicitarCompraService = new SolicitarCompraService(conexao);
      const itensSolicitacaoService = new
ItensSolicitacaoService(conexao);

      // Inserir solicitação principal
      const idSolicitacao = await
solicitarCompraService.inserirSolicitacaoCompra(idUsuario);

      // Inserir itens da solicitação
      await itensSolicitacaoService.inserirItensSolicitacao(idSolicitacao,
itens);

      // Confirmar transação
      await transacao.commit();
      console.log('Solicitação registrada com sucesso!');
    } catch (erro) {
      // Reverter transação em caso de erro
      await transacao.rollback();
      throw new Error('Erro ao registrar a solicitação: ' + erro.message);
    }
  }
}

module.exports = OrquestradorSolicitacao;
```

4 Chamando o serviço na rota

Usaremos o OrquestradorSolicitacao na rota de API para inserir uma solicitação de compras e seus itens, garantindo o correto controle de transação.

Rota de API

```
const express = require('express');
const OrquestradorSolicitacao = require('../services/OrquestradorSolicitacao');

const router = express.Router();
```

```
// Rota para registrar uma solicitação de compra
router.post('/register', async (req, res) => {
  const { id_usuario, itens } = req.body;

  try {
    await OrquestradorSolicitacao.registrarSolicitacao(id_usuario, itens);
    res.status(201).send('Solicitação registrada com sucesso!');
  } catch (erro) {
    res.status(500).send('Erro ao registrar solicitação: ' + erro.message);
  }
});

module.exports = router;
```

3. Fluxo detalhado

1. Orquestração no Nível de Serviço (Iniciar a transação):

- Um serviço central (OrquestradorSolicitacao) é responsável por:
 - Iniciar a transação criando uma conexão exclusiva.
 - Chamar métodos de serviços (SolicitarCompraService, ItensSolicitacaoService).
 - Confirmar ou reverter a transação com base no sucesso ou falha.

2. Separação de Responsabilidades (Chamar métodos separados):

- O SolicitarCompraService é responsável por inserir o registro principal (solicitacao_compra) e retorna o ID.
- O ItensSolicitacaoService utiliza o ID da solicitação de compras para inserir os itens relacionados.
- Ambos compartilham a conexão gerenciada pelo TransacaoBanco.

3. Gerenciamento de Transação (Confirmar ou reverter a transação):

- A classe (TransacaoBanco) gerencia as conexões e as transações. Esta classe contém os métodos para iniciar (beginTransaction), confirmar (commit) e reverter (rollback) as transações. Essa classe assegura que todas as operações em uma transação são consistentes:
 - Se todos os métodos forem bem-sucedidos, a transação será confirmada (commit).
 - Se algum método falhar, a transação será revertida (rollback).

4. Integrando com a API:

- Ao utilizar o `OrquestradorSolicitacao` são realizadas as operações na rota da API (`/register`).

Pontos Importantes de Implementação

Confirmação Atomizada: Se qualquer etapa falhar (exemplo: falha ao adicionar itens), a transação inteira será revertida para evitar inconsistências.

Flexibilidade: Os serviços (`SolicitarCompraService`, `ItensSolicitacaoService`) podem ser reutilizados em diferentes fluxos.

Benefícios do modelo

Isolamento de responsabilidades:

Isolamento das responsabilidades em classes menores e reutilizáveis. Cada classe (`SolicitarCompraService` e `ItensSolicitacaoService`) trata de uma responsabilidade específica.

Controle centralizado:

Controle centralizado do fluxo e das transações por meio do `OrquestradorSolicitacao` que executa as operações mantendo o controle da transação.

Reaproveitamento de código:

Melhor manutenção e flexibilidade para futuras alterações. Os métodos podem ser usados individualmente ou em diferentes fluxos.

Atividade Prática 1

Vamos implementar um cadastro de Solicitações de Compras, garantindo a integridade das operações por meio do controle de transação.

Inicializando o projeto

1. Por meio do terminal crie uma pasta para o projeto:

```
mkdir crud-transacao
cd crud-transacao
```

2. Inicialize o projeto utilizando npm:

(Será criado o arquivo `package.json`)

```
npm init -y
```

3. Instale as dependências:

```
npm install express mysql2
```

Configurando o banco de dados

1. Crie o banco de dados: Utilize o MySQL criando as seguintes tabelas:

```
CREATE TABLE USUARIOS (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  NOME VARCHAR(100) NOT NULL,
  EMAIL VARCHAR(100) UNIQUE NOT NULL,
  SENHA VARCHAR(100) NOT NULL,
  DATA_CRIACAO TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE PRODUTOS (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  NOME VARCHAR(100) NOT NULL,
  DESCRICAO TEXT,
  PRECO DECIMAL(10,2) NOT NULL,
  ESTOQUE INT NOT NULL,
  DATA_CRIACAO TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE SOLICITACAO_COMPRAS (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  ID_USUARIO INT NOT NULL
);

CREATE TABLE ITENS_SOLICITACAO (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  ID_SOLICITACAO INT NOT NULL,
  ID_PRODUTO INT NOT NULL,
  QUANTIDADE INT NOT NULL,
  PRECO_UNITARIO DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (ID_SOLICITACAO) REFERENCES SOLICITACAO_COMPRAS(ID)
);
```

2. Teste a conexão com o banco:

Crie um arquivo `db.js` para gerenciar a conexão com o banco:

```
const mysql = require('mysql2/promise');

const db = mysql.createPool({
  host: 'localhost',
  user: 'seu_usuario',
  password: 'sua_senha',
  database: 'seu_banco_de_dados'
});

module.exports = db;
```

Implementando o gerenciador de transação

Crie o arquivo `TransacaoBanco.js` para gerenciar transações:

```
const db = require('./db');

class TransacaoBanco {
  constructor() {
    this.connection = null;
  }

  async iniciarTransacao() {
    this.connection = await db.getConnection();
    await this.connection.beginTransaction();
  }

  async commit() {
    if (this.connection) {
      await this.connection.commit();
      this.connection.release();
    }
  }

  async rollback() {
    if (this.connection) {
      await this.connection.rollback();
      this.connection.release();
    }
  }
}

module.exports = TransacaoBanco;
```

Criando as classes de serviço

Solicitação de Compra (`SolicitarCompraService.js`):

```

class SolicitarCompraService {
  constructor(conexao) {
    this.conexao = conexao;
  }

  async inserirSolicitacaoCompra(idUsuario) {
    const query = 'INSERT INTO SOLICITACAO_COMPRAS (ID_USUARIO) VALUES (?)';
    const [result] = await this.conexao.query(query, [idUsuario]);
    return result.insertId;
  }
}

module.exports = SolicitarCompraService;

```

Itens da Solicitação (ItensSolicitacaoService.js):

```

class ItensSolicitacaoService {
  constructor(conexao) {
    this.conexao = conexao;
  }

  async inserirItensSolicitacao(idSolicitacao, itens) {
    const query = 'INSERT INTO ITENS_SOLICITACAO (ID_SOLICITACAO, ID_PRODUTO, QUANTIDADE, PRECO_UNITARIO) VALUES (?, ?, ?, ?)';

    for (const item of itens) {
      await this.conexao.query(query, [
        idSolicitacao,
        item.id_produto,
        item.quantidade,
        item.preco_unitario
      ]);
    }
  }
}

module.exports = ItensSolicitacaoService;

```

Criando o Orquestrador de transações

Crie o serviço principal (OrquestradorSolicitacao.js):

```

const TransacaoBanco = require('./TransacaoBanco');
const SolicitarCompraService = require('./SolicitarCompraService');
const ItensSolicitacaoService = require('./ItensSolicitacaoService');

class OrquestradorSolicitacao {
  static async registrarSolicitacao(idUsuario, itens) {
    const transacao = new TransacaoBanco();

    try {
      await transacao.iniciarTransacao();
      const conexao = transacao.connection;

```



```

        const solicitarCompraService = new SolicitarCompraService(conexao);
        const itensSolicitacaoService = new
ItensSolicitacaoService(conexao);

        const idSolicitacao = await
solicitarCompraService.inserirSolicitacaoCompra(idUsuario);
        await itensSolicitacaoService.inserirItensSolicitacao(idSolicitacao,
itens);

        await transacao.commit();
        console.log('Solicitação registrada com sucesso!');
    } catch (erro) {
        await transacao.rollback();
        throw new Error('Erro ao registrar solicitação: ' + erro.message);
    }
}

module.exports = OrquestradorSolicitacao;

```

Criando a rota que consumirá o serviço principal

Implemente a rota de API (routes.js):

```

const express = require('express');
const OrquestradorSolicitacao = require('./OrquestradorSolicitacao');

const router = express.Router();

router.post('/register', async (req, res) => {
    const { id_usuario, itens } = req.body;

    try {
        await OrquestradorSolicitacao.registrarSolicitacao(id_usuario, itens);
        res.status(201).send('Solicitação registrada com sucesso!');
    } catch (erro) {
        res.status(500).send('Erro ao registrar solicitação: ' + erro.message);
    }
});

module.exports = router;

```

Configurando o servidor

Crie o arquivo principal (server.js):

```

const express = require('express');
const router = require('./routes');

const app = express();
app.use(express.json());
app.use('/api', router);

```

```
const PORT = 3000;  
app.listen(PORT, () => console.log(`Servidor rodando na porta ${PORT}`));
```

Testando a solução

1. Execute o servidor:

```
node server.js
```

2. Faça um POST na rota `/api/register` com os seguintes dados em json:

(Use ferramentas como Postman para testar a API)

```
{  
  "id_usuario": 1,  
  "itens": [  
    { "id_produto": 1, "quantidade": 2, "preco_unitario": 10.50 },  
    { "id_produto": 2, "quantidade": 1, "preco_unitario": 15.00 }  
  ]  
}
```

Atividade Prática 2

Vamos incluir as operações necessárias para a alteração e exclusão das Solicitações de Compras, utilizando controle de transações.

Para incluir as operações de **editar** e **excluir** vamos precisar adicionar métodos específicos no serviço `SolicitarCompraService` e ajustar as rotas para suportar essas operações.

Atualizando novos métodos ao Serviço de Solicitação de Compras

Adicione métodos para editar (`atualizarSolicitacaoCompra`) e excluir (`excluirSolicitacaoCompra`) na classe `SolicitarCompraService`.

```
class SolicitarCompraService {  
  constructor(conexao) {  
    this.conexao = conexao;  
  }  
}
```

```

    async inserirSolicitacaoCompra(idUsuario) {
        const query = 'INSERT INTO SOLICITACAO_COMPRAS (ID_USUARIO) VALUES (?)';
        const [result] = await this.conexao.query(query, [idUsuario]);
        return result.insertId;
    }

    async atualizarSolicitacaoCompra(idSolicitacao, idUsuario) {
        const query = 'UPDATE SOLICITACAO_COMPRAS SET ID_USUARIO = ? WHERE ID
= ?';
        await this.conexao.query(query, [idUsuario, idSolicitacao]);
    }

    async excluirSolicitacaoCompra(idSolicitacao) {
        const query = 'DELETE FROM SOLICITACAO_COMPRAS WHERE ID = ?';
        await this.conexao.query(query, [idSolicitacao]);
    }
}

```

Adicionando os novos Métodos ao Orquestrador

Atualize o `OrquestradorSolicitacao` para incluir as operações de atualização e exclusão.

```

class OrquestradorSolicitacao {
    static async registrarSolicitacao(idUsuario, itens) {
        const transacao = new TransacaoBanco();

        try {
            await transacao.iniciarTransacao();
            const conexao = transacao.connection;

            const solicitarCompraService = new SolicitarCompraService(conexao);
            const itensSolicitacaoService = new
ItensSolicitacaoService(conexao);

            const idSolicitacao = await
solicitarCompraService.inserirSolicitacaoCompra(idUsuario);
            await itensSolicitacaoService.inserirItensSolicitacao(idSolicitacao,
itens);

            await transacao.commit();
            console.log('Solicitação registrada com sucesso!');
        } catch (erro) {
            await transacao.rollback();
            throw new Error('Erro ao registrar solicitação: ' + erro.message);
        }
    }

    static async atualizarSolicitacao(idSolicitacao, idUsuario) {
        const transacao = new TransacaoBanco();

        try {
            await transacao.iniciarTransacao();
            const conexao = transacao.connection;

            const solicitarCompraService = new SolicitarCompraService(conexao);
            await
solicitarCompraService.atualizarSolicitacaoCompra(idSolicitacao, idUsuario);

            await transacao.commit();
            console.log('Solicitação atualizada com sucesso!');
        } catch (erro) {

```

```

        await transacao.rollback();
        throw new Error('Erro ao atualizar a solicitação: ' + erro.message);
    }
}

static async excluirSolicitacao(idSolicitacao) {
    const transacao = new TransacaoBanco();

    try {
        await transacao.iniciarTransacao();
        const conexao = transacao.connection;

        const solicitarCompraService = new SolicitarCompraService(conexao);
        await
solicitarCompraService.excluirSolicitacaoCompra(idSolicitacao);

        await transacao.commit();
        console.log('Solicitação excluída com sucesso!');
    } catch (erro) {
        await transacao.rollback();
        throw new Error('Erro ao excluir a solicitação: ' + erro.message);
    }
}
}

module.exports = OrquestradorSolicitacao;

```

Ajustando as Rotas

No arquivo de rotas, adicione os endpoints para **atualizar** e **excluir**.

```

const express = require('express');
const OrquestradorSolicitacao = require('./OrquestradorSolicitacao');

const router = express.Router();

// Registrar uma solicitação de compra
router.post('/register', async (req, res) => {
    const { id_usuario, itens } = req.body;

    try {
        await OrquestradorSolicitacao.registrarSolicitacao(id_usuario, itens);
        res.status(201).send('Solicitação registrada com sucesso!');
    } catch (erro) {
        res.status(500).send('Erro ao registrar solicitação: ' + erro.message);
    }
});

// Atualizar uma solicitação de compra
router.put('/update/:id', async (req, res) => {
    const idSolicitacao = req.params.id;
    const { id_usuario } = req.body;

    try {
        await OrquestradorSolicitacao.atualizarSolicitacao(idSolicitacao,
id_usuario);
        res.status(200).send('Solicitação atualizada com sucesso!');
    } catch (erro) {
        res.status(500).send('Erro ao atualizar solicitação: ' + erro.message);
    }
});

```

```
// Excluir uma solicitação de compra
router.delete('/delete/:id', async (req, res) => {
  const idSolicitacao = req.params.id;

  try {
    await OrquestradorSolicitacao.excluirSolicitacao(idSolicitacao);
    res.status(200).send('Solicitação excluída com sucesso!');
  } catch (erro) {
    res.status(500).send('Erro ao excluir solicitação: ' + erro.message);
  }
});

module.exports = router;
```

Teste as Operações

A Solicitação de Compras deve suportar as operações:

Registrar uma solicitação (POST /api/register)

Atualizar uma solicitação (PUT /api/update/:id)

Excluir uma solicitação (DELETE /api/delete/:id)

(Caso ocorra qualquer erro, como falha de conexão ou erro de SQL, a transação será automaticamente revertida graças ao `rollback`)

Execute o servidor:

```
node server.js
```

Use ferramentas como Postman para testar os endpoints:

Inserir uma solicitação:

```
POST /api/register
{
  "id_usuario": 2,
  "itens": [
    { "id_produto": 1, "quantidade": 10, "preco_unitario": 1.00 },
    { "id_produto": 2, "quantidade": 12, "preco_unitario": 8.50 }
  ]
}
```

Atualizar uma solicitação:

```
PUT /api/update/1
{
  "id_usuario": 2
}
```

Excluir uma solicitação:

```
DELETE /api/delete/1
```